API Security Risk Analysis Report

Cyber Security Task 3 – Future Interns

Name: Shifa Tahreem

Internship: Cyber Security Internship

CIN: FIT/JUN26/CS9072

Date of Submission: June 2026

Assessment Type: API Security Risk Analysis

1. Executive Summary

This report presents a read-only API Security Risk Analysis conducted on the JSONPlaceholder public API. The objective of the assessment was to evaluate common API security risks, including authentication, authorization, data exposure, rate limiting, and input validation controls.

Testing was performed using Postman and public API endpoints without attempting exploitation or unauthorized access. The assessment identified several security concerns that, while acceptable in a public testing environment, could create risks if implemented in a production SaaS application.

2. Scope of Assessment

API Tested

API Name: JSONPlaceholder

API URL: https://jsonplaceholder.typicode.com

Endpoints Tested

- GET /users
- GET /users/1
- GET /posts
- GET /comments

Tools Used

- Postman
- Browser Developer Tools
- Microsoft Word
- GitHub

Assessment Methodology

The assessment was conducted using a read-only approach and included:

1. API documentation review
2. Endpoint testing
3. Response analysis
4. Header inspection

5. Authentication assessment
6. Authorization assessment
7. Security risk identification
8. Remediation recommendation development

3.Risk Overview

A total of five security observations were identified during the assessment.

Severity Distribution:

• Medium: 3 Findings

• Low-Medium: 1 Finding

• Low: 1 Finding

No critical or high-severity issues were identified during the assessment.

The most significant concerns were the absence of authentication controls, excessive exposure of user-related information, and the lack of visible rate-limiting mechanisms. While these characteristics are expected in a public testing API, they would present security concerns if observed in a production SaaS environment.

4. Findings Summary

Finding ID	**Risk**	**Severity**
F-01	Missing Authentication Controls	Medium
F-02	Excessive Data Exposure	Medium
F-03	Missing Rate Limiting Evidence	Medium
F-04	Predictable Resource Access	Low-Medium
F-05	Input Validation Not Evident	Low

5.Alignment with OWASP API Security Risks

The identified findings were mapped against common API security concerns described by the OWASP API Security Top 10 project.

Finding ID	Related OWASP API Security Concern
F-01	Broken Authentication
F-02	Excessive Data Exposure / Broken Object Property Level Authorization
F-03	Unrestricted Resource Consumption
F-04	Broken Object Level Authorization (Potential Risk)
F-05	Security Misconfiguration / Improper Input Validation

6. Detailed Findings

F-01: Missing Authentication Controls

Severity: Medium

Observation:

The tested API endpoints returned data successfully without requiring authentication credentials, API keys, session tokens, or bearer tokens.

Examples:

- GET /users
- GET /users/1
- GET /posts
- GET /comments

Business Impact

In a production environment, unauthenticated access may allow unauthorized users to retrieve information that should be protected. This increases the risk of information disclosure and misuse of API resources.

Remediation

Implement authentication controls such as:

- OAuth 2.0
- JWT-based authentication
- API Keys
- Session-based authentication where appropriate

F-02: Excessive Data Exposure

Severity: Medium

Observation:

The user endpoint returned multiple data fields, including:

- Name
- Username
- Email
- Address
- Phone Number
- Website
- Company Information

Applications may not require all of these fields for normal functionality.

Business Impact

Exposing more information than necessary increases privacy risks and expands the attack surface available to malicious users.

Remediation

Apply data minimization principles and return only the information required by the client application.

F-03: Missing Rate Limiting Evidence

Severity: Medium

Observation

No visible evidence of rate-limiting controls was identified during testing. The API responses did not indicate restrictions on request frequency.

Business Impact

Without rate limiting, attackers may abuse API resources through excessive requests, resulting in degraded performance, service disruption, or increased operational costs.

Remediation

Implement:

- API rate limiting

- Request throttling
- API gateway protections
- Abuse detection mechanisms

F-04: Predictable Resource Access

Severity: Low-Medium

Observation:

Individual resources were accessible through sequential identifiers.

Examples:

- /users/1
- /users/2
- /users/3

The resource identifiers followed a predictable pattern.

Business Impact

In production environments, predictable identifiers may increase the risk of unauthorized access to other users' records if authorization checks are not properly enforced.

Remediation

Implement strong authorization controls and verify user permissions before granting access to resources.

F-05: Input Validation Not Evident

Severity: Low

Observation:

Input validation mechanisms were not clearly visible during testing and were not explicitly documented in the tested endpoints.

Business Impact

Insufficient input validation can increase the likelihood of malformed requests and may expose applications to injection-related attacks in production systems.

Remediation

Implement robust server-side validation and maintain clear documentation of accepted input formats and validation rules.

7. Evidence

The following evidence was collected during testing:

i. Users Endpoint Response

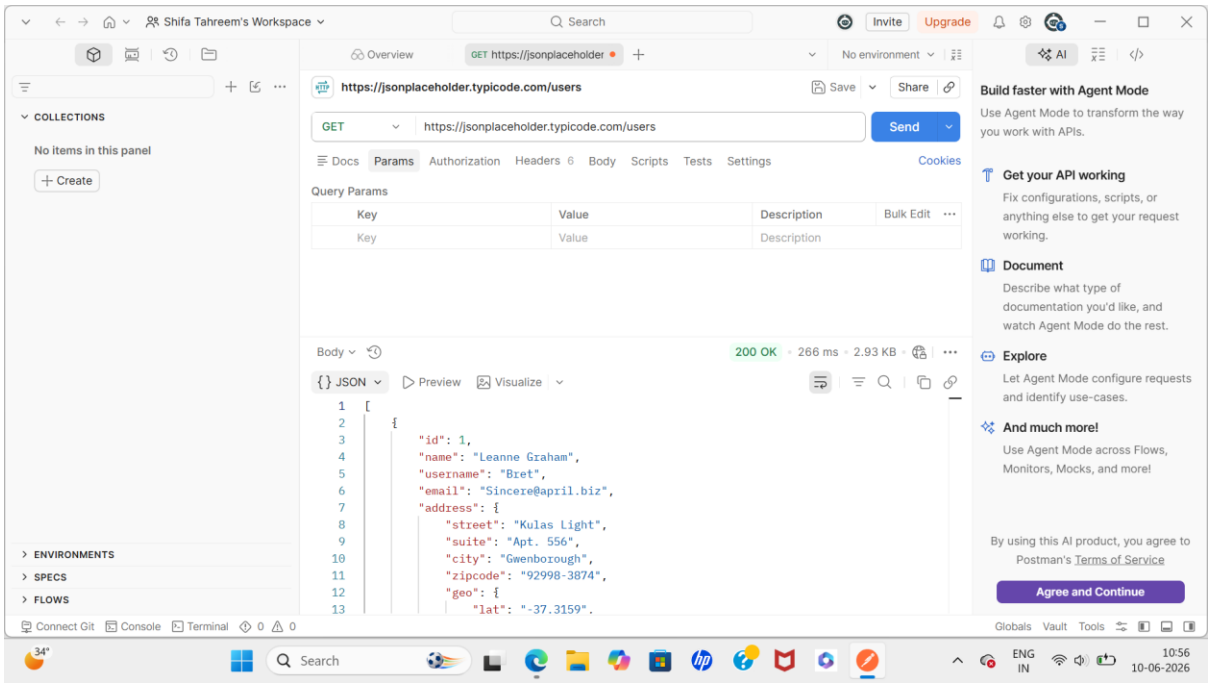

Figure 1: Response from GET /users endpoint

ii. Single User Endpoint Response

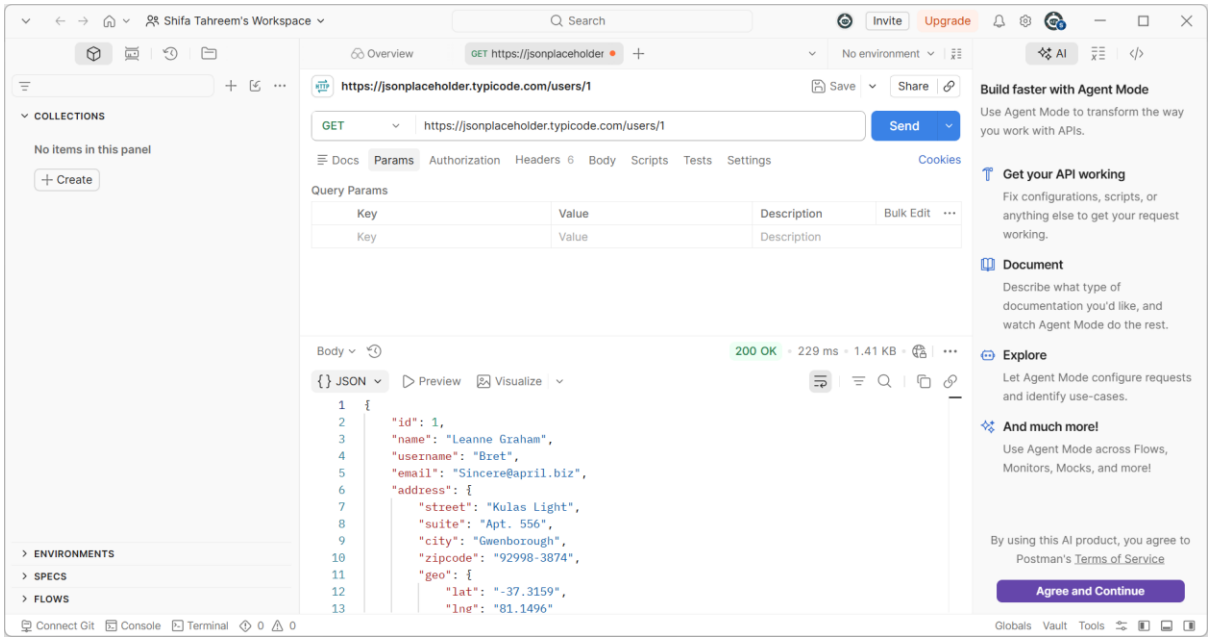

Figure 2: Response from GET /users/1 endpoint

iii. Headers Observation

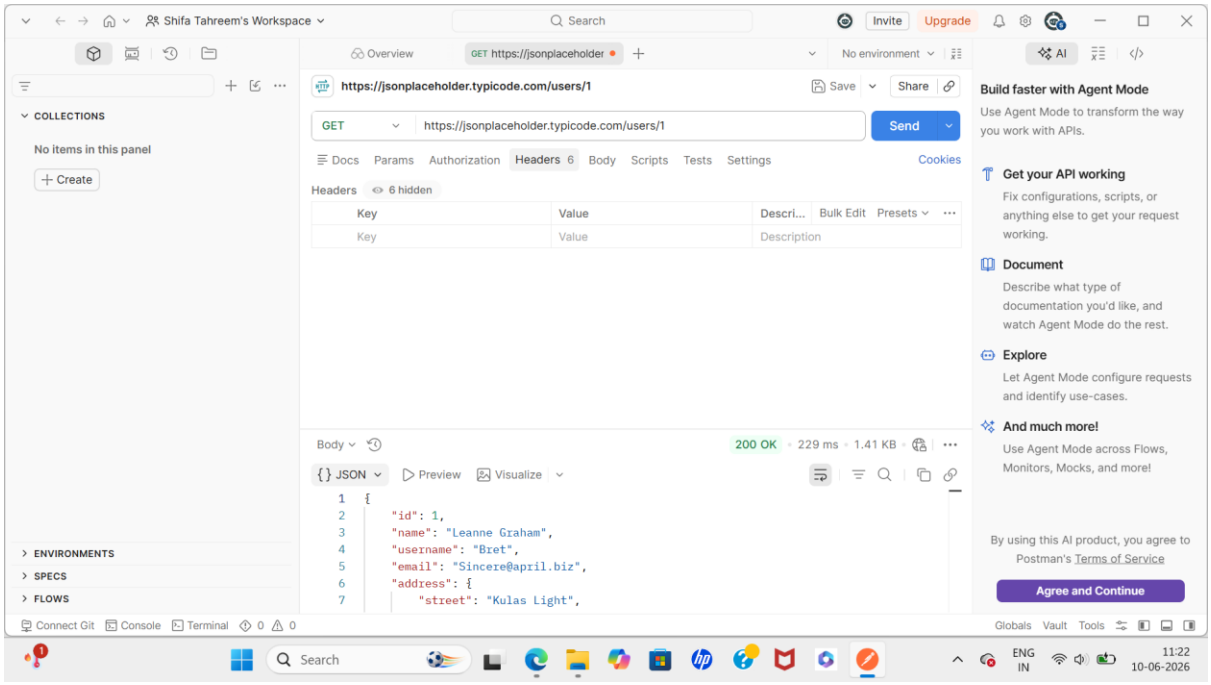

Figure 3: Request header analysis performed using Postman

iv. Posts Endpoint Response

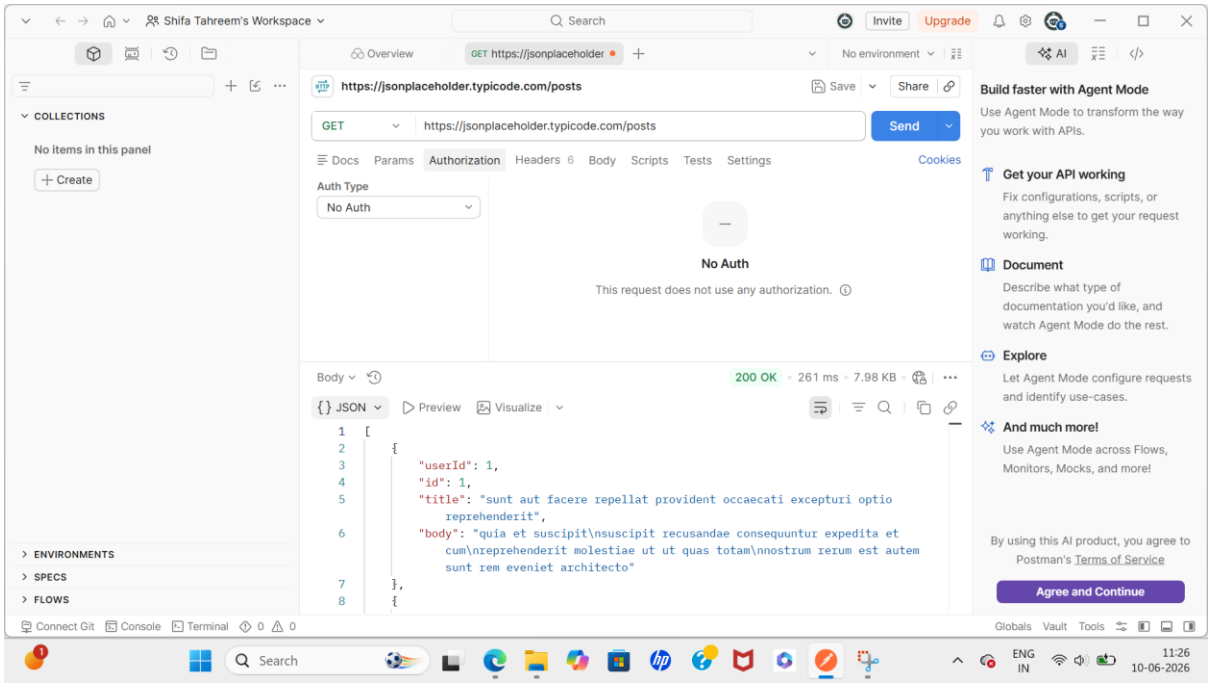

Figure 4: Response from GET /posts endpoint

v. Comments Endpoint Response

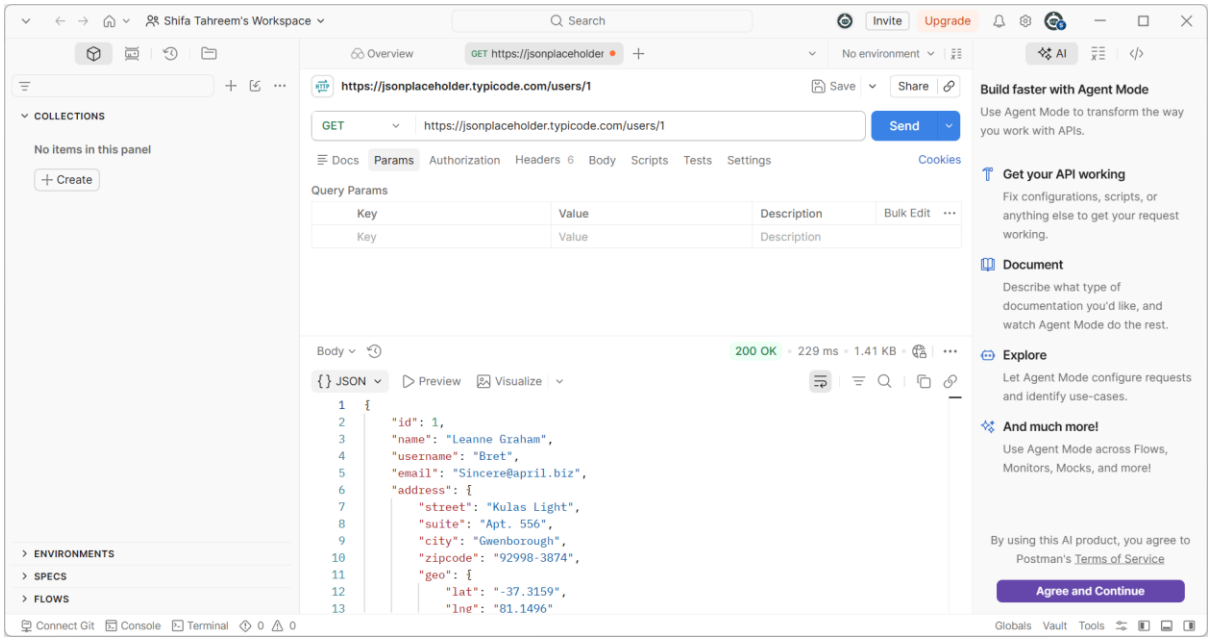

Figure 5: Response from GET /comments endpoint

8. Conclusion

The assessment demonstrated that the JSONPlaceholder API is intentionally designed as a public testing platform and therefore does not implement several security controls that would normally be expected in production environments.

Five security observations were identified relating to authentication, data exposure, resource access patterns, rate limiting, and input validation. While these observations do not represent vulnerabilities within the intended purpose of the platform, they illustrate common API security weaknesses that organizations should avoid when developing production SaaS applications.

Implementing strong authentication mechanisms, enforcing authorization checks, minimizing exposed data, validating user input, and introducing rate-limiting controls would significantly improve the security posture of a production API.

9. Disclaimer

This assessment was conducted exclusively on a publicly available testing API using read-only techniques. No exploitation, bypass attempts, denial-of-service testing, or unauthorized activities were performed.